



UNIVERSIDAD TECNOLOGIA DE HONDURAS

Asignatura:

Computación en la nube

Trabajo:

Actividad # 3- Servicio en la nube, detección de enfermedades hoja del café

Catedrático:

Asalia Zavala

Estudiante:

Astrid Lizbeth Castellanos Pineda

Cuenta:

202410010300

Sección:

1

Fecha de entrega:

9 de agosto del 2026

Detección de Enfermedades en Hojas de Café

Documentación del proyecto — Computación en la Nube

Introducción

Este proyecto lo hicimos para la clase de Computación en la Nube. La idea era construir un servicio web que le sirva de verdad a un caficultor: le sube una foto de una hoja de café y el sistema le dice si tiene alguna enfermedad, con qué nivel de confianza, y qué puede hacer al respecto. Combinamos un modelo de visión artificial entrenado por nosotros con la API de Groq para que las recomendaciones no salgan de una lista fija, sino redactadas según lo que se detectó en cada caso.

Arquitectura del sistema

La aplicación es una app de Streamlit que corre como un solo servicio web. Cuando el usuario sube una imagen, la app la redimensiona a 224x224 y la normaliza con el preprocesamiento de MobileNetV2, para que quede en el mismo formato que se usó durante el entrenamiento. Esa imagen se le pasa al modelo (modelo_cafe.h5), que devuelve la clase más probable y el porcentaje de confianza.

Con esa clase detectada, la app arma un prompt y se lo manda a la API de Groq, que redacta en español la descripción de la enfermedad, las recomendaciones técnicas, las buenas prácticas de cultivo y las acciones de seguimiento. Ese texto se muestra en pantalla organizado en secciones y, aparte, se genera un PDF descargable con reportlab, para que el caficultor se quede con el registro de esa muestra.

Todo el flujo pasa dentro de un solo proceso de Streamlit: no hay backend ni base de datos separados, el modelo se carga una sola vez en memoria (con `cache_resource`) y queda disponible mientras la app esté corriendo.

Servicios en la nube utilizados

Usamos Google Colab con GPU para entrenar el modelo, porque entrenar MobileNetV2 con fine-tuning en una computadora normal hubiera tardado muchísimo más. Ahí subimos el dataset, entrenamos, evaluamos y descargamos el modelo (.h5) y el archivo de clases (labels.json).

Para el despliegue de la app usamos Streamlit Community Cloud, que se conecta directo al repositorio de GitHub: cada vez que se hace push, la app se vuelve a desplegar sola. La API key de Groq no va en el código, se guarda en la sección de Secrets de Streamlit Cloud, así que no queda expuesta en el repositorio.

La inteligencia para redactar las recomendaciones no la ponemos nosotros, la consumimos como servicio: la API de Groq corre el modelo de lenguaje (llama-3.3-70b-versatile) en su propia infraestructura y nosotros solo le mandamos el prompt y recibimos la respuesta.

Tecnologías empleadas

Python es el lenguaje base de todo el proyecto. Para el modelo usamos TensorFlow / Keras, con transfer learning sobre MobileNetV2 preentrenado en ImageNet: le quitamos la capa final, le pusimos una capa densa propia y entrenamos primero solo esa parte, y después hicimos fine-tuning descongelando las últimas 30 capas con una tasa de aprendizaje baja.

El frontend y el servidor de la app son Streamlit. Para las recomendaciones técnicas usamos el SDK de groq en Python, y para armar el PDF descargable usamos reportlab. El manejo de imágenes lo hacemos con Pillow y numpy. Todo el control de versiones y el despliegue está conectado a través de Git y GitHub.

Flujo de funcionamiento del sistema

1. El usuario entra a la app y sube una foto de una hoja de café (o la toma con la cámara).
2. Presiona “Analizar muestra” y la imagen se redimensiona y normaliza para que quede lista para el modelo.
3. El modelo (modelo_cafe.h5) predice la clase y devuelve el porcentaje de confianza.
4. La app arma un prompt con la clase detectada y lo envía a la API de Groq.
5. Groq devuelve el texto con las 4 secciones (descripción, manejo preventivo, buenas prácticas y seguimiento), que la app separa y muestra en pantalla.
6. El usuario puede descargar todo ese diagnóstico como PDF, generado al vuelo con reportlab.

Alcance y limitaciones

Algo que hay que dejar claro: el modelo solo reconoce 4 clases (hoja sana, minador, phoma y roya). El dataset original traía también muestras de cercospora y araña roja, pero esas carpetas venían con archivos corruptos y no se pudieron usar, así que esas dos clases quedaron fuera del entrenamiento. Además, ningún modelo de este tipo llega al 100% de precisión real; lo que hicimos para que sea lo más confiable posible fue usar aumento de datos, dividir entrenamiento/validación y hacer fine-tuning, pero la app siempre debe mostrarse como una herramienta de apoyo, no como un diagnóstico definitivo, y por eso también recomienda confirmar con un técnico agrónomo.